

LAB 01:

```
1  -- LAB 01:
2  EXPLAIN (ANALYZE, BUFFERS)
3  SELECT c.first_name, c.last_name, r.rental_date
4  FROM customer c
5  Join rental r ON c.customer_id = r.customer_id
6  WHERE c.customer_id = 1
```

Data Output Messages Notifications

Showing rows: 1 to 13

	QUERY PLAN
1	Nested Loop (cost=0.28..359.16 rows=32 width=21) (actual time=0.191..3.284 rows=32 loops=1)
2	Buffers: shared hit=1 read=152
3	-> Index Scan using customer_pkey on customer c (cost=0.28..8.29 rows=1 width=17) (actual time=0.026..0.043 rows=1 loo...
4	Index Cond: (customer_id = 1)
5	Buffers: shared hit=1 read=2
6	-> Seq Scan on rental r (cost=0.00..350.55 rows=32 width=10) (actual time=0.163..3.233 rows=32 loops=1)
7	Filter: (customer_id = 1)
8	Rows Removed by Filter: 16012
9	Buffers: shared read=150
10	Planning:
11	Buffers: shared hit=204 read=5 dirtied=2
12	Planning Time: 4.892 ms
13	Execution Time: 3.321 ms

Outer table: customer

Inner table: rental

Nested loop is chosen because the outer side only has 1 row, the formula yields a very small result - far cheaper than a Hash Join, which would require building a hash table over the entire 'rental' table

loops =1 because the outer table (customer) has already been filtered down to 1 row, the inner loop (scanning rental) only needs to execute once for that row

LAB 02:

```

14  -- LAB 02:
15  EXPLAIN (ANALYZE, BUFFERS)
16  SELECT f.title, l.name as language
17  FROM film f
18  JOIN language l ON f.language_id = l.language_id;

```

Data Output Messages Notifications

Showing rows:

QUERY PLAN	
	text
1	Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=0.344..0.646 rows=1000 loops=1)
2	Hash Cond: (f.language_id = l.language_id)
3	Buffers: shared hit=88 read=1
4	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=0.018..0.204 rows=1000 loo...
5	Buffers: shared hit=88
6	-> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=0.318..0.318 rows=6 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB
8	Buffers: shared read=1
9	-> Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=0.308..0.310 rows=6 loo...
10	Buffers: shared read=1
11	Planning:
12	Buffers: shared hit=295 read=1
13	Planning Time: 6.841 ms
14	Execution Time: 0.697 ms

- 'language' table is being hashed
- 'film' table is being probed
- Memory usage is 9kB
- After SET work_mem = '64kB';

```

14  -- LAB 02:
15  SET work_mem = '64kB';
16  EXPLAIN (ANALYZE, BUFFERS)
17  SELECT f.title, l.name as language
18  FROM film f
19  JOIN language l ON f.language_id = l.language_id;

```

Data Output Messages Notifications

Showing rows: 1

QUERY PLAN	
	text
1	Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=0.021..0.215 rows=1000 loops=1)
2	Hash Cond: (f.language_id = l.language_id)
3	Buffers: shared hit=89
4	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=0.010..0.089 rows=1000 loops=1)
5	Buffers: shared hit=88
6	-> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=0.007..0.007 rows=6 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB
8	Buffers: shared hit=1
9	-> Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=0.004..0.005 rows=6 loops=1)
10	Buffers: shared hit=1
11	Planning Time: 0.124 ms
12	Execution Time: 0.248 ms

LAB 03:

1.

```

29 SET enable_hashjoin = OFF;
30 SET enable_nestloop = OFF;
31
32 EXPLAIN (ANALYZE , BUFFERS)
33 SELECT f.film_id , f.title, i.inventory_id
34 FROM film f
35 JOIN inventory i ON f.film_id = i.film_id
36 ORDER BY f.film_id ;

```

Data Output Messages Notifications

Showing rows: 1 to 17

	QUERY PLAN
	text
1	Merge Join (cost=412.64..622.36 rows=4581 width=23) (actual time=4.141..5.148 rows=4581 loops=1)
2	Merge Cond: (f.film_id = i.film_id)
3	Buffers: shared hit=111 read=1, temp read=10 written=12
4	-> Index Scan using film_pkey on film f (cost=0.28..127.33 rows=1000 width=19) (actual time=0.004..0.177 rows=1000 loops=1)
5	Buffers: shared hit=83 read=1
6	-> Materialize (cost=412.37..435.27 rows=4581 width=6) (actual time=4.132..4.553 rows=4581 loops=1)
7	Buffers: shared hit=28, temp read=10 written=12
8	-> Sort (cost=412.37..423.82 rows=4581 width=6) (actual time=4.127..4.320 rows=4581 loops=1)
9	Sort Key: i.film_id
10	Sort Method: external merge Disk: 80kB
11	Buffers: shared hit=28, temp read=10 written=12
12	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.012..0.259 rows=4581 loops=1)
13	Buffers: shared hit=25
14	Planning:
15	Buffers: shared hit=90 read=1

It used an index scan using film_pkey on film f .Since film_id is the primary key of film (already indexed/ordered), the film side typically gets its data presorted for free via an Index Scan, with no Sort node needed

2.

12	RESET ALL;
13	
14	EXPLAIN(ANALYZE,BUFFERS)
15	SELECT f.film_id, f.title, i.inventory_id
16	FROM film f
17	JOIN inventory i ON f.film_id = i.film_id ;
Data Output Messages Notifications	
Showing rows: 1 to 14	
	QUERY PLAN text
1	Hash Join (cost=110.50..193.39 rows=4581 width=23) (actual time=3.357..4.005 rows=4581 loops=1)
2	Hash Cond: (i.film_id = f.film_id)
3	Buffers: shared hit=79 read=34
4	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.007..0.141 rows=4581 loo...
5	Buffers: shared hit=25
6	-> Hash (cost=98.00..98.00 rows=1000 width=19) (actual time=3.338..3.339 rows=1000 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 59kB
8	Buffers: shared hit=54 read=34
9	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=19) (actual time=0.005..3.259 rows=1000 loo...
10	Buffers: shared hit=54 read=34
11	Planning:
12	Buffers: shared hit=6
13	Planning Time: 0.221 ms
14	Execution Time: 4.101 ms

It switched back to Hash Join because without a sort requirement, Merge Join lost its main advantage (getting sorted output for free). On a small dataset like dvdrental, hashing the smaller table is simply cheaper than paying for a Sort step. Merge Join only wins when data is already sorted via an index, or when the dataset is too large to fit in memory for a Hash Join.

Hands-on lab

Task 1

21	SET enable_indexscan = off ;
22	EXPLAIN ANALYZE SELECT * FROM rental WHERE rental_date > '2005-05-24' ;
Data Output Messages Notifications	
Showing rows: 1	
	QUERY PLAN text
1	Seq Scan on rental (cost=0.00..350.55 rows=16044 width=36) (actual time=0.366..3.337 rows=16044 loop...
2	Filter: (rental_date > '2005-05-24 00:00:00':timestamp without time zone)
3	Planning Time: 11.778 ms
4	Execution Time: 3.590 ms

24	SET enable_indexscan = on ;
25	EXPLAIN ANALYZE SELECT * FROM rental WHERE rental_date > '2005-05-24' ;
Data Output Messages Notifications	
≡ + 📄 ▼ 📋 ▼ 🗑️ 🗑️ 🗑️ 📄 ⬇️ 📈 SQL Showing rows: 1	
	QUERY PLAN text 🔒
1	Seq Scan on rental (cost=0.00..350.55 rows=16044 width=36) (actual time=0.009..0.857 rows=16044 loop...
2	Filter: (rental_date > '2005-05-24 00:00:00'::timestamp without time zone)
3	Planning Time: 0.102 ms
4	Execution Time: 1.100 ms

The plan stayed Seq Scan in both cases because the filter isn't selective (16,044 rows matched - too many for an index to help). The faster execution time on the second run is just caching, not a plan change.

Task 2:

Before analyze:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT c.first_name, c.last_name, r.rental_date, i.inventory_id
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
JOIN inventory i ON r.inventory_id = i.inventory_id;
```

	QUERY PLAN	
	text	
1	Hash Join (cost=150.55..545.57 rows=16044 width=25) (actual time=1.027..4.833 rows=16044 loops=1)	
2	Hash Cond: (r.inventory_id = i.inventory_id)	
3	Buffers: shared hit=177 read=7	
4	-> Hash Join (cost=22.48..375.33 rows=16044 width=25) (actual time=0.575..2.948 rows=16044 loops=1)	
5	Hash Cond: (r.customer_id = c.customer_id)	
6	Buffers: shared hit=152 read=7	
7	-> Seq Scan on rental r (cost=0.00..310.44 rows=16044 width=14) (actual time=0.009..0.527 rows=16044 loop...	
8	Buffers: shared hit=150	
9	-> Hash (cost=14.99..14.99 rows=599 width=17) (actual time=0.558..0.558 rows=599 loops=1)	
10	Buckets: 1024 Batches: 1 Memory Usage: 39kB	
11	Buffers: shared hit=2 read=7	
12	-> Seq Scan on customer c (cost=0.00..14.99 rows=599 width=17) (actual time=0.007..0.504 rows=599 loo...	
13	Buffers: shared hit=2 read=7	
14	-> Hash (cost=70.81..70.81 rows=4581 width=4) (actual time=0.431..0.431 rows=4581 loops=1)	
15	Buckets: 8192 Batches: 1 Memory Usage: 226kB	
16	Buffers: shared hit=25	
17	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=4) (actual time=0.007..0.187 rows=4581 loops=...	
18	Buffers: shared hit=25	
19	Planning:	
20	Buffers: shared hit=88 read=17 dirtied=1	
21	Planning Time: 9.925 ms	
22	Execution Time: 5.178 ms	

After analyze:

```

ANALYZE customer;
ANALYZE rental;
ANALYZE inventory;

EXPLAIN (ANALYZE, BUFFERS)
SELECT c.first_name, c.last_name, r.rental_date, i.inventory_id
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
JOIN inventory i ON r.inventory_id = i.inventory_id;

```

	QUERY PLAN	
	text	
1	Hash Join (cost=150.55..545.57 rows=16044 width=25) (actual time=0.575..4.311 rows=16044 loops=1)	
2	Hash Cond: (r.inventory_id = i.inventory_id)	
3	Buffers: shared hit=184	
4	-> Hash Join (cost=22.48..375.33 rows=16044 width=25) (actual time=0.113..2.417 rows=16044 loops=1)	
5	Hash Cond: (r.customer_id = c.customer_id)	
6	Buffers: shared hit=159	
7	-> Seq Scan on rental r (cost=0.00..310.44 rows=16044 width=14) (actual time=0.006..0.489 rows=16044 loop...	
8	Buffers: shared hit=150	
9	-> Hash (cost=14.99..14.99 rows=599 width=17) (actual time=0.101..0.101 rows=599 loops=1)	
10	Buckets: 1024 Batches: 1 Memory Usage: 39kB	
11	Buffers: shared hit=9	
12	-> Seq Scan on customer c (cost=0.00..14.99 rows=599 width=17) (actual time=0.004..0.044 rows=599 loo...	
13	Buffers: shared hit=9	
14	-> Hash (cost=70.81..70.81 rows=4581 width=4) (actual time=0.442..0.442 rows=4581 loops=1)	
15	Buckets: 8192 Batches: 1 Memory Usage: 226kB	
16	Buffers: shared hit=25	
17	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=4) (actual time=0.003..0.187 rows=4581 loops=...	
18	Buffers: shared hit=25	
19	Planning:	
20	Buffers: shared hit=53	
21	Planning Time: 0.334 ms	
22	Execution Time: 4.626 ms	

Hash Join was chosen. After ANALYZE, Plan didn't change because the statistics of the database was corrected before analyzing.